BVMT AI Platform

Sprint 1 Days 9-10 + Full Project Roadmap

Esprit School of Business — Tunisia | March 2026

Where You Are Right Now

You are at Sprint 1 Day 8 completed, Day 9 starting. Here is your honest status before reading the next steps:

Component	Status	Result
daily_prices	~120,000 rows, 68 stocks, 2016-2025	✓ DONE
company_metadata	68 rows	✓ DONE
computed_indicators	MA5/20/50, RSI14, volatility — all stocks	✓ DONE
news_articles	7,937 articles, ilboursa.com, 68 stocks, 2016-2026	✓ DONE
pdf_metadata	815 PDFs discovered + downloaded, 906 MB on disk	✓ DONE
financial_statements	~172 rows so far — batch still running (FY only)	⏳ IN PROGRESS
Agent files	agent_state.py, base_agent.py, orchestrator.py, data_agent.py	✓ DONE
MCP servers	database_server.py, files_server.py — skeletons exist	⏳ IN PROGRESS
TFT model	Not started — Sprint 2 task	📋 NEXT
FundamentalAgent (XGBoost)	Not started — Sprint 3 task	📋 NEXT
SentimentAgent (FinBERT)	Not started — Sprint 3 task	📋 NEXT
GraphAgent (GAT)	Not started — Sprint 4 task	📋 NEXT
RiskAgent (Monte Carlo)	Not started — Sprint 5 task	📋 NEXT
XAIAgent (SHAP)	Not started — Sprint 5 task	📋 NEXT
FastAPI app	Not started — Sprint 6 task	📋 NEXT

Full Project Roadmap — 60 Days

This is your complete roadmap from Day 9 to the final demo. Each sprint builds on the last — do not skip ahead.

Sprint 1 — Days 1 to 10 (Foundation)

- **DONE —** Days 1-5
  - Price data loaded, indicators computed, agent files created, pipeline tested
- **DONE —** Days 6-8
  - 815 PDFs downloaded from CMF, financial extraction pipeline working, DataAgent upgraded
- **NEXT —** Day 9
  - Sprint 1 test suite: 5 pytest tests + verify financial_statements quality
- **NEXT —** Day 10
  - GitHub commit, Sprint 2 plan, Lightning AI account setup

Sprint 2 — Days 11 to 20 (Technical Analysis — TFT Model)

- Days 11-12: Build feature engineering notebook on your local machine
  - Create feature_engineering.ipynb — combines price + indicators into model input
  - Output: feature_vectors table in PostgreSQL (68 stocks x 10 years)
- Days 13-14: Upload to Lightning AI and train TFT
  - Free T4 GPU on lightning.ai — no laptop GPU needed
  - Training script: training/train_tft.py — already exists in your project
  - Target: 80%+ directional accuracy on validation set
- Days 15-17: Train CNN-LSTM (comparison model for thesis)
  - Same data, different architecture — shows TFT advantage
  - Walk-forward cross-validation: Academic Contribution #2
- Days 18-20: Build TechnicalAgent
  - Loads model from /models/, runs inference, returns prediction + confidence
  - Plugs into OrchestratorAgent pipeline

Sprint 3 — Days 21 to 30 (Fundamental + Sentiment Agents)

- Days 21-23: FundamentalAgent
  - XGBoost on 15+ financial ratios extracted from your 815 PDFs
  - Health score 0-100 for each company, classification: STRONG/MODERATE/WEAK/CRITICAL
  - Input: financial_statements table (already being populated)
- Days 24-26: SentimentAgent
  - FinBERT (French) on French news articles from ilboursa.com
  - CAMeLBERT (Arabic) on Arabic articles if present
  - Output: sentiment score per article -> aggregated per stock per week
- Days 27-30: MCP servers — make them real
  - database_server.py: fill in all tool functions (postgres_query, timescale_range_query)
  - files_server.py: fill in PDF reader, web scraper tools
  - model_server.py: fill in run_tft, run_xgboost inference tools

Sprint 4 — Days 31 to 40 (Graph Neural Network)

- Days 31-34: Build correlation graph
  - Compute stock-to-stock correlation matrix from daily_prices
  - Store in correlation_matrix table
  - Edge weight = Pearson correlation over 252-day window
- Days 35-38: Train GAT (Graph Attention Network)
  - Academic Contribution #1: first GAT application on BVMT data
  - Nodes = stocks, edges = correlations, features = TFT outputs
  - Train on Lightning AI — T4 GPU
- Days 39-40: GraphAgent
  - Runs GAT inference, detects ripple effects between stocks
  - Plugs into OrchestratorAgent after TechnicalAgent + FundamentalAgent

Sprint 5 — Days 41 to 50 (Risk + Explanation)

- Days 41-44: RiskAgent
  - Monte Carlo simulation: 10,000 paths per stock, Student-t distribution
  - Outputs: VaR (95%, 99%) and CVaR for each stock
  - Takes ~2 seconds per stock — fast enough for real-time use
- Days 45-47: XAIAgent
  - SHAP values on XGBoost output: which ratio drove the score
  - Jinja2 templates: generate plain-language explanation in French/Arabic
  - Example: 'BIAT received a STRONG score mainly because its NPL ratio fell to 7.2%'
- Days 48-50: Knowledge Hub (ModerationAgent)
  - Community forum for investors — posts stored in hub_posts table
  - ModerationAgent flags spam, financial misinformation, and off-topic content

Sprint 6 — Days 51 to 60 (API + Demo + Thesis)

- Days 51-54: FastAPI application
  - Endpoints: GET /analysis/{ticker}, GET /risk/{ticker}, POST /hub/post
  - Celery task queue for background model inference
  - MLflow tracking for model experiment logging
- Days 55-57: End-to-end evaluation
  - Run full pipeline on 10 stocks — measure latency, accuracy, edge cases
  - Compare TFT vs CNN-LSTM on out-of-sample data (2025 Q1)
  - Document all three academic contributions with metrics
- Days 58-60: Thesis writeup + demo
  - Chapter 4 (Technical): architecture diagram, agent interaction diagrams
  - Chapter 5 (Results): accuracy tables, SHAP plots, VaR charts
  - Live demo: run OrchestratorAgent on AMEN BANK, show full analysis output

---

DAY 9 — Sprint 1 Test Suite + Financial Quality Review (3-4 hours)

Day 9 has two goals: (1) verify that the financial data extracted so far is usable, and (2) write 5 pytest tests that confirm your entire Sprint 1 pipeline works end to end. You do this WHILE the batch continues running — both tasks are independent.

Part A — Verify financial_statements quality (1 hour)

The batch is extracting FY_ANNUAL PDFs using two parallel notebooks (Groq and GitHub/GPT-4o). Before the batch finishes, you can already check what came in and fix any obvious errors.

Step 1 — Run the verification query in pgAdmin

Open pgAdmin, connect to bvmt_db, open the Query Tool, and run this SQL:

```
SELECT ticker, period, total_assets, equity, pnb, net_result,
       npl_ratio, extraction_confidence, needs_review
FROM financial_statements
ORDER BY ticker, period
LIMIT 60;
```

What to look for in the output — 3 sanity checks:

- Sanity check 1 — total_assets must be larger than net_result. If any row shows net_result > total_assets, that company had a wrong unit (DT instead of kDT). Note the ticker name.
- Sanity check 2 — equity must be less than total_assets. If equity > total_assets, something is wrong.
- Sanity check 3 — for banks, pnb should not be NULL. Banks always have a Produit Net Bancaire. If a bank row shows pnb = NULL, the P&L page was missed.

Step 2 — Fix wrong-unit companies

From your previous session, ATL and ATTIJARI LEASING had unit errors and were deleted. The batch will re-extract them with the correct DT unit because of the LEASING_TICKERS fix in Cell 2. If you see other non-bank companies with unrealistic values, run this in pgAdmin to divide by 1000:

```
UPDATE financial_statements
SET
    total_assets = total_assets / 1000,
    equity = equity / 1000,
    net_result = net_result / 1000,
    revenue = revenue / 1000,
    extraction_notes = extraction_notes || ' | manual_div1000'
WHERE ticker = 'TICKER_NAME_HERE';
```

Step 3 — Count what you have

Run this query to see a summary by company type:

```
SELECT company_type,
       COUNT(*) AS total_records,
       COUNT(*) FILTER (WHERE total_assets IS NOT NULL) AS has_assets,
       ROUND(AVG(extraction_confidence)::numeric, 3) AS avg_conf,
       COUNT(*) FILTER (WHERE needs_review = TRUE) AS needs_fix
FROM financial_statements
GROUP BY company_type;
```

Expected at this point: banks should show avg_conf around 0.75-0.88, non-banks around 0.50-0.75. If a type shows avg_conf < 0.4, there may be a systematic extraction issue to investigate.

Part B — Write the Sprint 1 test suite (2 hours)

Create a new file tests/test_sprint1.py. This file already exists as a skeleton from Day 5 — you are now filling it in with real tests. You need exactly 5 tests:

#	Action	File / Location	Why it matters
T1	test_prices_loaded	tests/test_sprint1.py	Confirms daily_prices has 68+ stocks and 100,000+ rows
T2	test_indicators_computed	tests/test_sprint1.py	Confirms computed_indicators has RSI and MA values (not all NULL)
T3	test_news_loaded	tests/test_sprint1.py	Confirms news_articles has 5,000+ rows and recent articles
T4	test_financials_available	tests/test_sprint1.py	Confirms financial_statements has data for at least 5 banks
T5	test_pipeline_runs	tests/test_sprint1.py	Runs the full OrchestratorAgent on AMEN BANK and checks output shape

Complete code for tests/test_sprint1.py

Copy this entire file into tests/test_sprint1.py and save it:

```
import pytest
import asyncio
import os
import sys
from pathlib import Path
from dotenv import load_dotenv

load_dotenv()
sys.path.insert(0, str(Path(__file__).parent.parent))

import psycopg2
from agents.orchestrator import OrchestratorAgent
from agents.agent_state import AgentState
```

```
def get_conn():
    return psycopg2.connect(
        host=os.getenv('DB_HOST'), port=int(os.getenv('DB_PORT', 5432)),
        dbname=os.getenv('DB_NAME'), user=os.getenv('DB_USER'),
        password=os.getenv('DB_PASSWORD'))
```

```
# T1 — price data
def test_prices_loaded():
    conn = get_conn()
    with conn.cursor() as cur:
        cur.execute('SELECT COUNT(*) FROM daily_prices')
        total = cur.fetchone()[0]
        cur.execute("SELECT COUNT(DISTINCT ticker) FROM daily_prices")
        stocks = cur.fetchone()[0]
    conn.close()
    assert total >= 100_000, f'Expected 100k+ rows, got {total}'
    assert stocks >= 60,     f'Expected 60+ stocks, got {stocks}'
```

```
# T2 — indicators
def test_indicators_computed():
    conn = get_conn()
    with conn.cursor() as cur:
        cur.execute("""
            SELECT COUNT(*) FROM computed_indicators
            WHERE rsi_14 IS NOT NULL AND ma_20 IS NOT NULL""")
        ok = cur.fetchone()[0]
    conn.close()
    assert ok >= 50_000, f'Expected 50k+ indicator rows, got {ok}'
```

```
# T3 — news
def test_news_loaded():
    conn = get_conn()
    with conn.cursor() as cur:
        cur.execute('SELECT COUNT(*) FROM news_articles')
        total = cur.fetchone()[0]
        cur.execute("""
            SELECT COUNT(*) FROM news_articles
            WHERE published_at >= '2024-01-01'""")
        recent = cur.fetchone()[0]
    conn.close()
    assert total >= 5_000,  f'Expected 5k+ news rows, got {total}'
    assert recent >= 200,   f'Expected 200+ recent articles, got {recent}'
```

```
# T4 — financials
def test_financials_available():
    conn = get_conn()
    with conn.cursor() as cur:
        cur.execute("""
            SELECT COUNT(DISTINCT ticker) FROM financial_statements
            WHERE company_type = 'bank'
            AND total_assets IS NOT NULL""")
        banks = cur.fetchone()[0]
```

```
    conn.close()
    assert banks >= 5, f'Expected 5+ banks with financial data, got {banks}'
```

```
# T5 — full pipeline
def test_pipeline_runs():
    orch = OrchestratorAgent()
    result = asyncio.run(orch.run('AMEN BANK'))
    assert result is not None
    assert 'data_quality_score' in result or hasattr(result, 'data_quality_score')
```

How to run the tests

Open your VS Code terminal (make sure your virtual environment is active), then run:

```
cd C:\Users\Negza\Desktop\projects\pfe\bvmt_project
python -m pytest tests/test_sprint1.py -v
```

Expected output: 5 passed. If T4 fails because you have fewer than 5 banks, that is acceptable — the batch is still running. Move on and the test will pass by Day 10 when more records are extracted.

DAY 10 — GitHub Commit + Sprint 2 Preparation + Lightning AI Setup (3 hours)

Day 10 is your sprint close-out day. You commit all Sprint 1 work to GitHub, set up Lightning AI for model training in Sprint 2, and write your Sprint 2 plan. This is also when you accept that the financial batch will continue running in the background across Days 10-15 — it does not block Sprint 2.

Part A — GitHub commit (30 minutes)

If you do not have a Git repository yet, initialize one now:

```
cd C:\Users\Negza\Desktop\projects\pfe\bvmt_project
git init
git add .gitignore requirements.txt
git add agents/ mcp_servers/ sql/ training/ tests/
git add main.py
# DO NOT add: .env, data/, models/ (too large / sensitive)
git commit -m 'Sprint 1 complete: data pipeline + agent foundation'
git remote add origin https://github.com/YOUR_USERNAME/bvmt-project.git
git push -u origin main
```

Create a .gitignore file if you do not have one. It must contain these lines:

```
.env
data/
models/
__pycache__/
*.pyc
.venv/
*.ipynb_checkpoints
```

Part B — Lightning AI account setup (30 minutes)

Lightning AI is the free cloud GPU platform where you will train your TFT and GAT models. Your laptop GPU is not needed for training. Here is how to set it up:

- Go to lightning.ai and sign up with your student email
- Click New Studio — select the Python environment
- You get a free T4 GPU with 16 GB VRAM — more than enough for your models
- Upload your training script: training/train_tft.py
- Upload your data: the feature CSV will be exported from PostgreSQL in Sprint 2 Days 11-12

What you do NOT need to do today: you are not training anything yet. You just need the account ready so that when Day 13 arrives you can go directly to training without setup delays.

Part C — Sprint 2 planning (1 hour)

Open a new file sprint2_plan.md in your project root and write the following. This is not just documentation — it helps you think through what data format the TFT model needs before you start building.

What the TFT model needs as input

The Temporal Fusion Transformer (TFT) predicts stock direction 7 days ahead. It needs these input columns per stock per day:

- Static (company-level, fixed): ticker encoded as integer, sector code, company_type (bank / non_bank / insurance)
- Time-varying past (historical): cloture (closing price), daily_return, ma_5, ma_20, rsi_14, volatility_20, volume
- Target: direction = 1 if cloture shifted 7 days forward > cloture today, else 0
- Time index: seance date converted to integer (days since 2016-01-01)

This means your Day 11-12 task is to join daily_prices + computed_indicators + company_metadata into one flat DataFrame, compute the target variable, and export it as a CSV for Lightning AI.

Part D — Review financial batch status (30 minutes)

At the end of Day 10, check how many records have been extracted so far. Run this in pgAdmin:

```
SELECT
    COUNT(*) AS total_records,
    COUNT(DISTINCT ticker) AS companies_covered,
    COUNT(*) FILTER (WHERE total_assets IS NOT NULL) AS has_assets,
    COUNT(*) FILTER (WHERE extraction_confidence >= 0.75) AS high_quality,
    COUNT(*) FILTER (WHERE needs_review = TRUE) AS needs_review
FROM financial_statements;
```

By Day 10 you should have at least 150-200 records. By Day 15 the batch should be mostly complete. You do not need to wait for the batch — FundamentalAgent in Sprint 3 will use whatever is available, and the needs_review records will be fixed manually in a short Day 7-review session.

DAY 11 — Feature Engineering — Build TFT Input Dataset (3-4 hours)

Day 11 is the first day of Sprint 2. You build the feature dataset that goes into the TFT model. Everything runs locally on your machine — no GPU needed yet. This is a Jupyter notebook task.

What you will build: notebooks/feature_engineering.ipynb

This notebook queries your PostgreSQL database, joins 3 tables, and exports a clean CSV for Lightning AI training. Here is the step-by-step plan:

#	Action	File / Location	Why it matters
1	Query daily_prices for all 68 stocks (2016-2025)	PostgreSQL -> pandas	Raw OHLCV data for all stocks
2	Join with computed_indicators on (seance, isin_code)	Inner join	Adds MA, RSI, volatility columns
3	Join with company_metadata for sector/type labels	Left join on isin_code	Adds static features (bank/non_bank)
4	Compute target variable: direction_7d	pandas shift(-7)	What TFT will predict
5	Encode tickers as integers (label encoding)	pandas factorize	TFT needs numeric group IDs
6	Filter: require at least 252 days of history per stock	groupby filter	Remove stocks with insufficient data
7	Export to data/features/tft_features.csv	~8 million rows expected	Upload to Lightning AI
8	Verify: check shape, nulls, class balance	Print summary	Catch problems before training

Key formula — target variable

The target variable direction_7d is what the model learns to predict. It is 1 if the stock goes up in 7 days and 0 if it goes down or stays flat:

```
df['future_price'] = df.groupby('ticker')['cloture'].shift(-7)
df['direction_7d'] = (df['future_price'] > df['cloture']).astype(int)
df = df.dropna(subset=['direction_7d'])  # remove last 7 rows per stock
```

Important note about class balance

Tunisian stocks tend to have more up-days than down-days in long bull market periods. Check the balance:

```
print(df['direction_7d'].value_counts(normalize=True))
```

If the result shows more than 60/40 split, you will need to apply class weighting during training. Note this result and keep it for your thesis — it is a market microstructure observation about BVMT.

Thesis Integration Points

As you work through each sprint, these are the specific moments where you should capture results for your thesis writeup. Keep a research_notes.md file and update it after each of these:

Sprint	Academic Contribution	What to Record	Where in Thesis
S2	TFT vs CNN-LSTM on BVMT (Contribution #2)	Accuracy, F1, Sharpe ratio on 2025 hold-out	Chapter 5, Table 5.1
S1	Adaptive weight rebalancing (Contribution #3)	Weight table: bank (0.42/0.31/0.27) vs non-bank (0.59/0.16/0.27)	Chapter 4, Section 4.3
S4	First GAT on BVMT (Contribution #1)	Which stocks have highest influence score, sector clusters	Chapter 5, Section 5.4
S5	Monte Carlo VaR validation	Compare VaR vs actual losses, backtesting results	Chapter 5, Section 5.5

Practical Notes for the Coming Days

Financial batch — what to do while it runs

- Let both notebooks (Groq and GPT-4o) run in parallel — they skip each other's records automatically
- Do not touch financial_statements manually unless you see a row with net_result > total_assets
- By Day 15 you should have 500+ records — enough for FundamentalAgent training in Sprint 3
- The remaining records (low confidence or failed) can be fixed in Sprint 3 Day 21 before training

When to stop worrying about the extraction

Stop checking the batch daily after Day 14. You only need financial data for companies that have enough price history AND appear frequently in news. Those are the 15-20 main stocks: AMEN BANK, BIAT, BNA, STB, BH, ATTIJARI BANK, ATB, UBCI, UIB, SFBT, POULINA, ONE TECH, DELICE HOLDING, CARTHAGE CEMENT, ARTES. If those 15 stocks have clean financial data, you have enough to train FundamentalAgent.

The 15 most important stocks for your project

Ticker	Sector	Why Important	Data Status (target)
AMEN BANK	Banking	Largest private bank, most liquid	10 FY records needed
BIAT	Banking	Largest bank by assets	10 FY records needed
BNA	Banking	State bank, high NPL ratio	10 FY records needed
STB	Banking	State bank, turnaround story	10 FY records needed
ATTIJARI BANK	Banking	Moroccan subsidiary, modern	10 FY records needed
ATB	Banking	Arab Tunisian Bank, stable	10 FY records needed
SFBT	F&B Beverages	Most liquid non-bank	10 FY records needed
POULINA	Conglomerate	Largest group, 30+ subsidiaries	10 FY records needed
ONE TECH	Tech/Export	Electronics, growth story	10 FY records needed
DELICE HOLDING	Food	Dairy group, IFRS reporting	10 FY records needed
ARTES	Auto	Vehicle distribution, DT unit	10 FY records needed
CARTHAGE CEMENT	Industrial	Cement, cyclical exposure	10 FY records needed
UBCI	Banking	French parent (BNP Paribas)	10 FY records needed
UIB	Banking	Societe Generale subsidiary	10 FY records needed
BH	Banking	Banque de l'Habitat, mortgages	10 FY records needed